



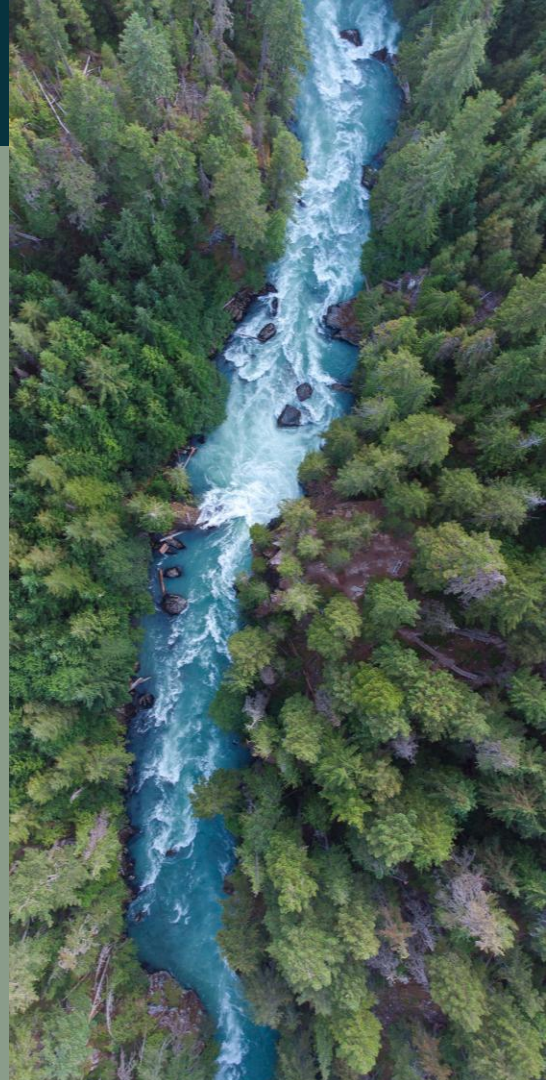
Natural Resources
Canada

Ressources naturelles
Canada

Geospatial Deep Learning in Practice

Victor Alhassan

Canada



Environment Setup in 2026

Primary Tools

Core



Development



Machine Learning



Environment Setup



Lightning ^{AI}

Creators of PyTorch Lightning



My Studios

1



My Inference



My Pipelines



My Jobs



My Published



My Quests

- Useful links



github.com/geoaiclassroom

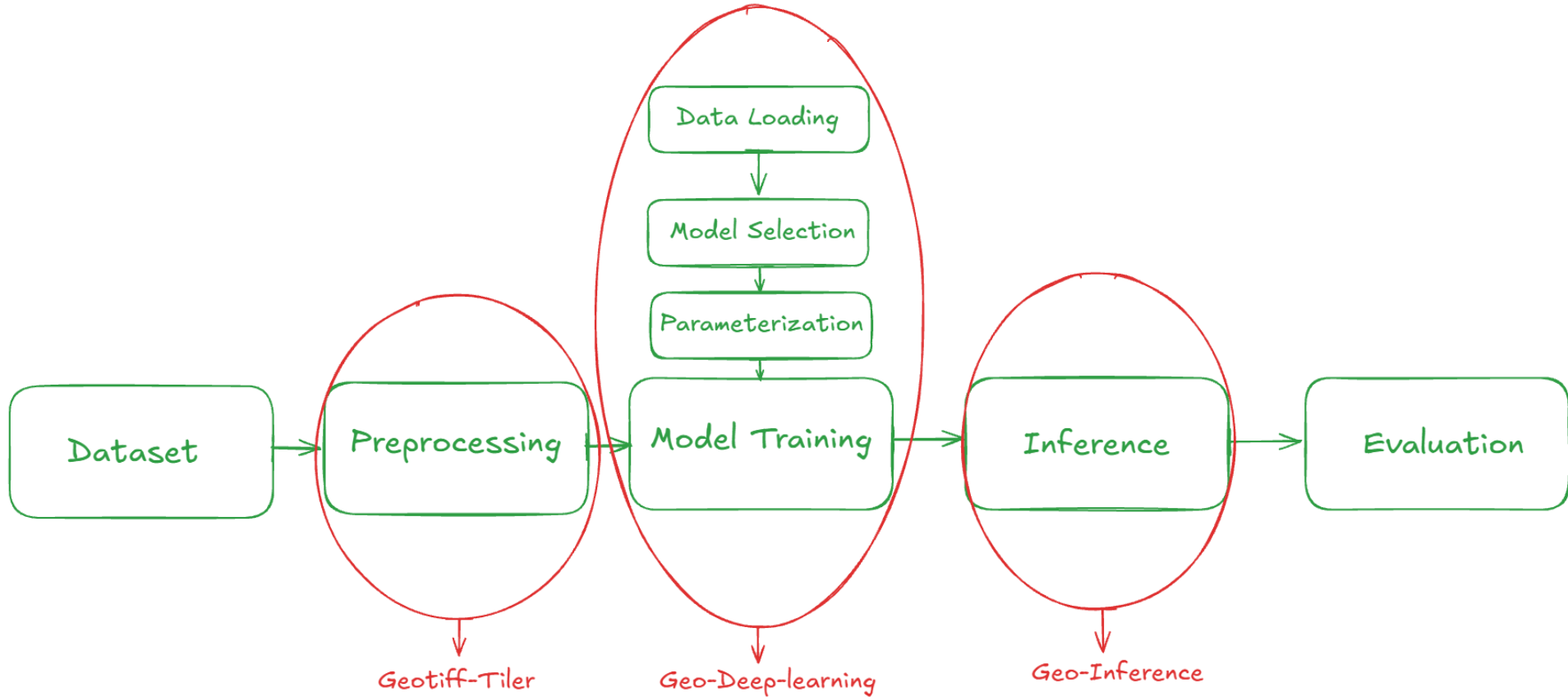


lightning.ai/geoi-classroom



lightning.ai/valhassan/templates/geoi

GeoAI Overview



Data Acquisition and Preprocessing

Data Acquisition

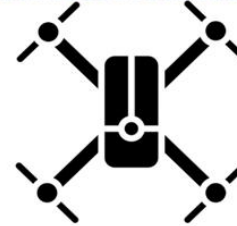
Platforms



Satellite



Airborne



UAV

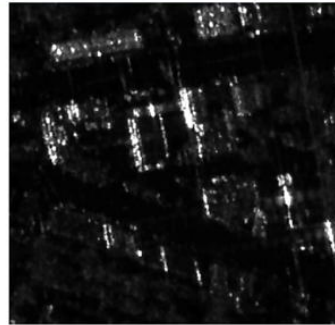
Icon images: Flaticon.com



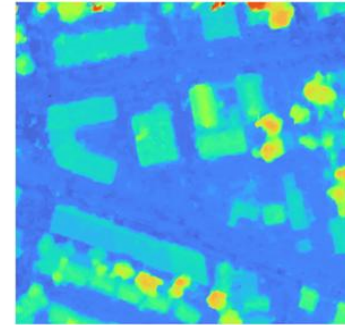
Multimodal data



Optical imagery

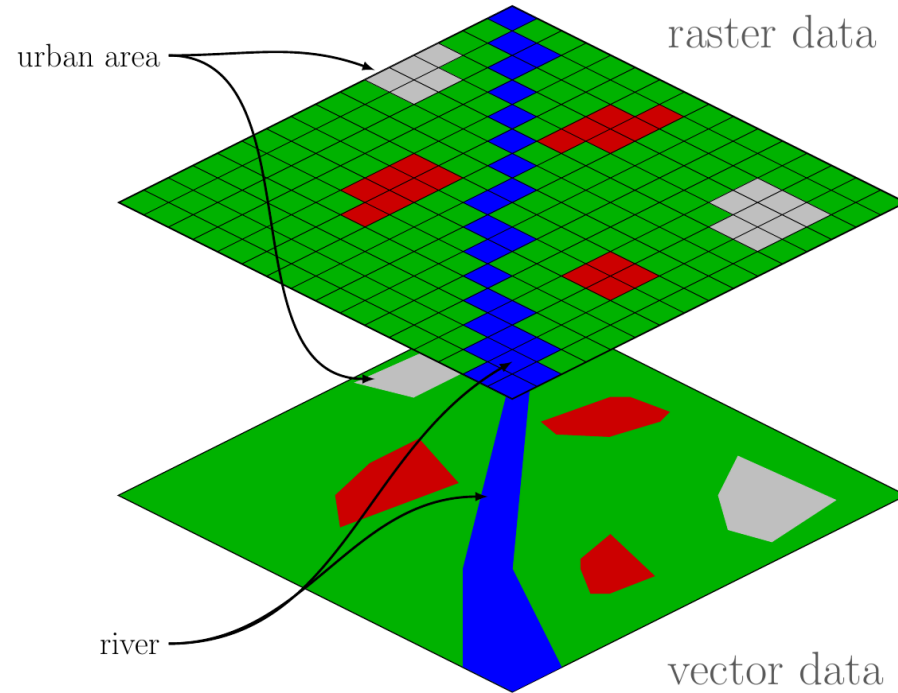


SAR imagery

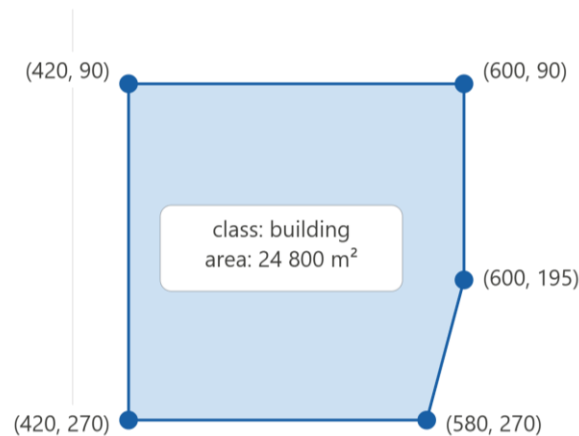
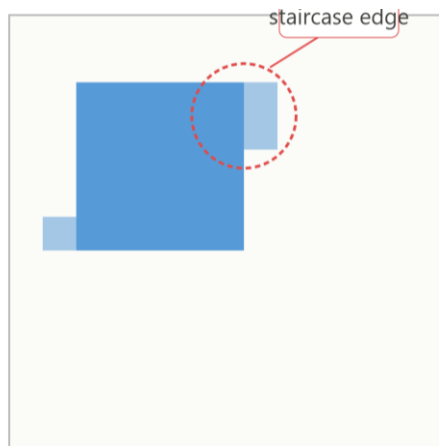


Elevation model

Data Format



Vector or Raster Format for Labels



A grid of integers	Geometric shapes (polygons, lines, points)
Staircase edges – one pixel defined by pixel size	Precise edges – Exact coordinates
Derived Target features are imprecise	Source of Truth – Derive Precise Target Features
Difficult to correct labels	Easy to correct labels – Move or add vertices

Data Preprocessing

geotiff-tiler

Spatial QA

- CRS alignment
- Overlap check
- Intersection clip
- Nodata masking

Patch Extraction

- Sliding window
- Border fill
- Empty filter
- Val split

Output

- tar shards (WebDataset)
- GeoTIFF +
- CSV index

Label Preparation

- Vector → raster
- Class mapping
- Edge erosion

Auxiliary Targets

- EDT
- Boundary
- SDF

Dataset Stats

- Per-band mean / std
- Class dist.
- Manifest

sample - VAL Split (CSV)

sample_ns2_small_rgbn_clahe25_corr_1024_1024



Label | 5 classes



roads_centerline_weight



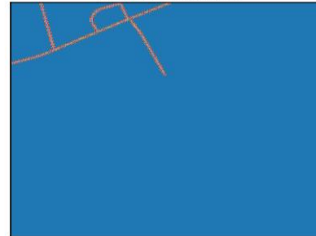
sample_ns2_small_rgbn_clahe25_corr_2048_1024



Label | 4 classes



roads_centerline_weight



sample_ns2_small_rgbn_clahe25_corr_4096_0



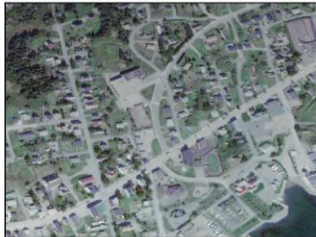
Label | 5 classes



roads_centerline_weight



sample_ns2_small_rgbn_clahe25_corr_2048_0



Label | 5 classes

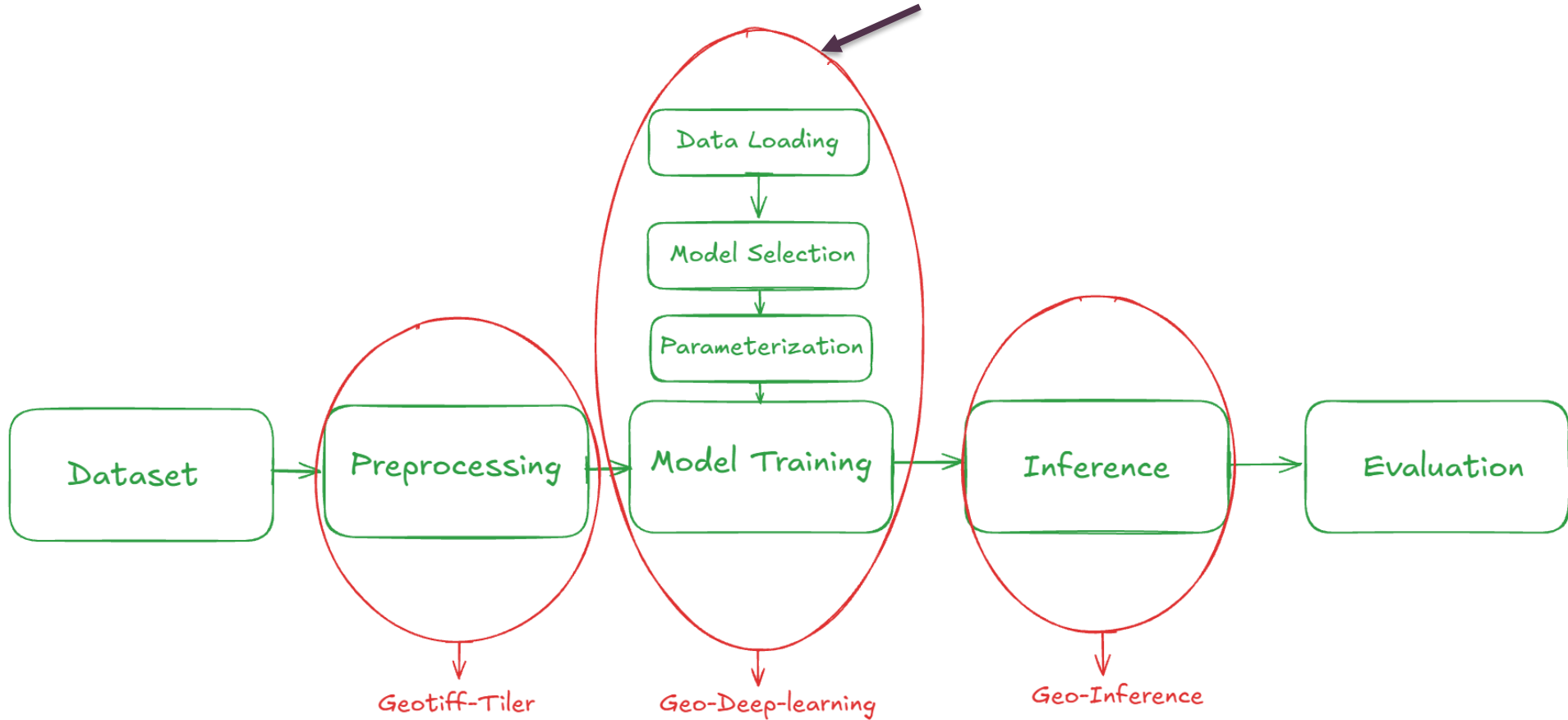


roads_centerline_weight



Modeling (geo-deep-learning)

GeoAI Overview



Mental Model of GDL

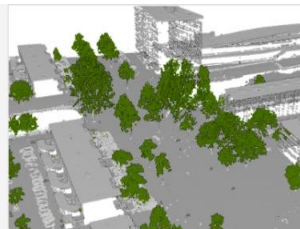
- **Geo:** Geospatial data in various forms; satellite images, aerial photos, and vector data (points, lines). These are our inputs.
- **Deep-Learning:** Deep learning is a family of algorithms that learn patterns from large datasets to optimize or solve tasks.

Geospatial Tasks

- Classification (What is this area? forest or water?)
- Detection (Where are the houses, fires, urban changes?)
- Segmentation (Pixel-level mapping of land cover or flood zones)
- Prediction (Weather forecast, likelihood of natural disasters)



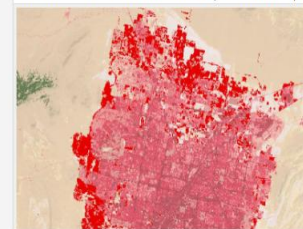
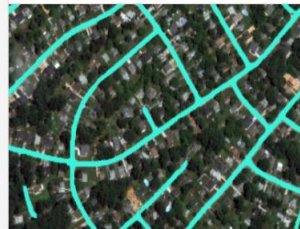
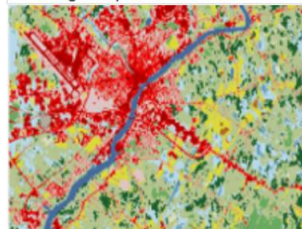
Building Footprint Extraction - USA



Tree Point Classification



Land Cover Classification (Sentinel-2)



GDL Framework

Why we built GDL?

Because practitioners need a modular, composable, extensible framework to build deep learning models for geospatial tasks.

The goal: To provide a clean, unified training framework for small and large geospatial vision models.

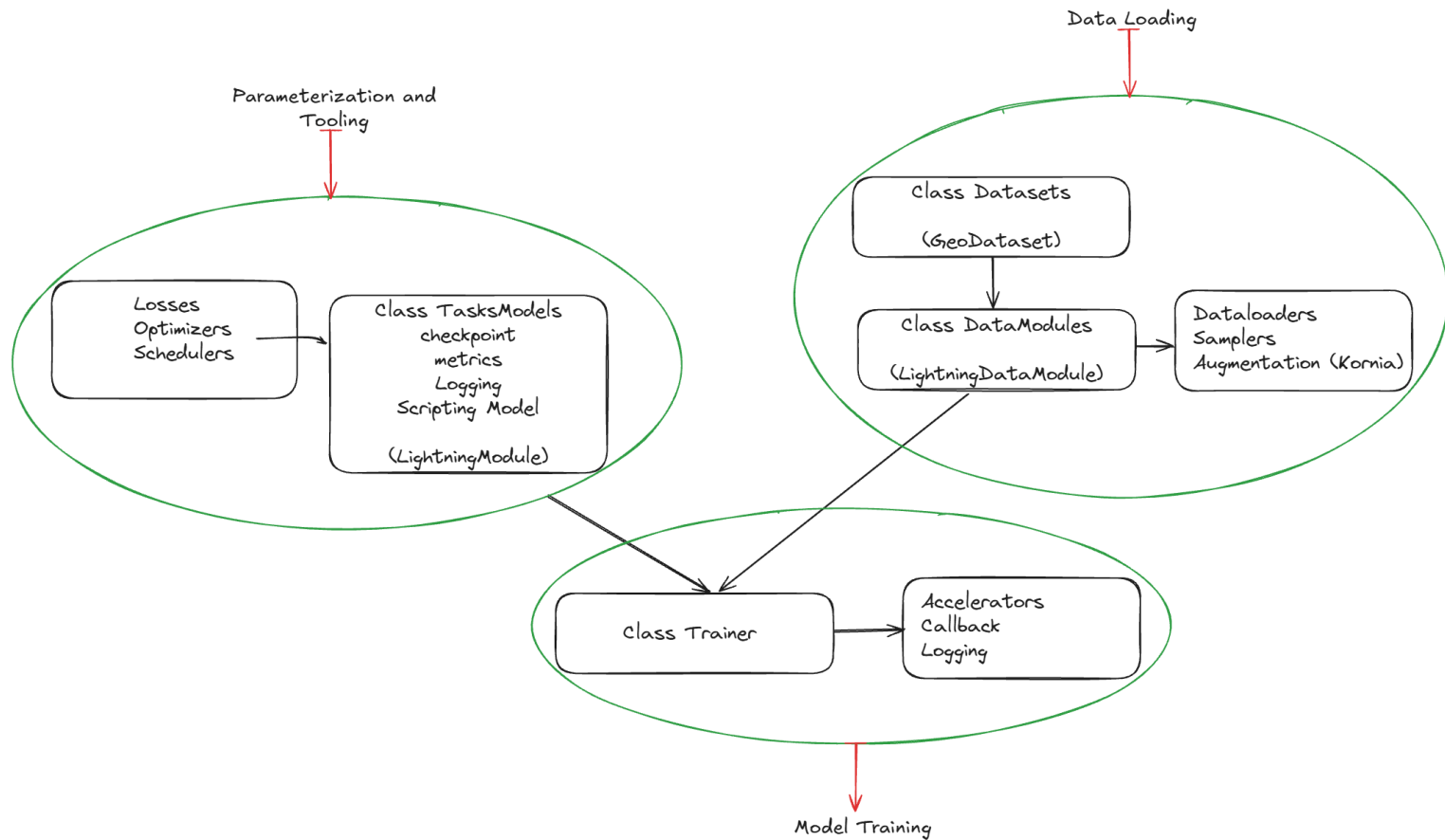


PyTorch



PyTorch-Lightning

Practical Components (GDL)



Data Loading

Datasets

- Datasets are the main entry point into the whole pipeline.
- It loads patches, normalize them, attach metadata, and hand them off to the **DataModule**.

Data Loading

Dataset Class	Description	Key Features
CSVDataset	Simple, local, easy to inspect. Ideal for prototyping or small patch-based datasets	Reads image + mask paths from a CSV file
ShardedDataset (WebDataset)	Scalable, multi-sensor, HPC-ready. Designed for large EO workloads.	Reads sharded `.tar` WebDataset files. Supports images, labels, metadata Loads sensor-specific normalization stats from JSON Automatically distributes shards across GPUs/nodes in DDP

Data Loading

DataModules

DataModules are the orchestration layer between Datasets and the Trainer.

Datasets = what the data is

DataModules = how the data flows

GDL provides complementary datamodules to the datasets.

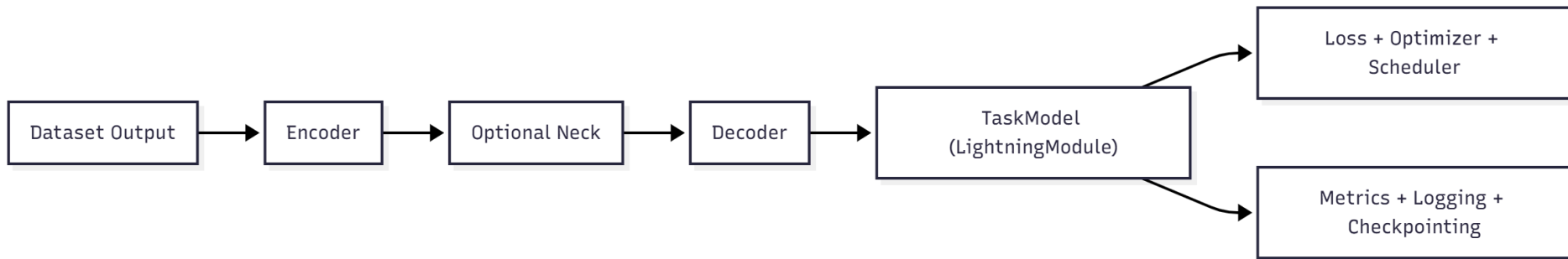
DataModule	Purpose	What it Does	Key Features
CSVDataModule	Simple, folder-based loading for small/local experiments	Instantiates 3 **CSVDatasets** (train/val/test) Wraps them with standard PyTorch DataLoaders	Train shuffling Multi-worker loading Prefetching, pin-memory, persistent workers
MultiSensorDataModule	Scalable, multi-sensor, HPC-oriented workloads	Uses <code>`create_sensor_datasets()`</code> to build multiple WebDatasets Uses WebLoader for train/val	Sensor mixing via <code>`RandomMix()`</code> DDP-aware shard splitting WebDataset-based batching Shuffling at shard + sample levels

Modeling

Models

This is where the learning happens.

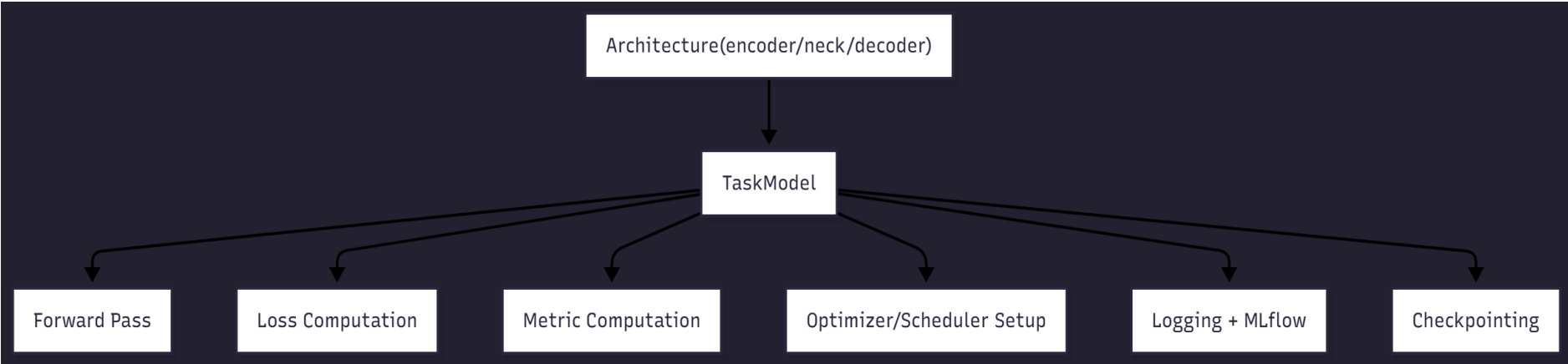
This component defines the model into a single, modular unit ready for training.



Model Structure/Architecture

Component	Purpose	Examples
Encoder	Extracts features from EO imagery	ResNet, MixTransformer, DOFA Encoder, Dinov3
Neck	Optional intermediate fusion or multi-scale processing	MultiLevelNeck (connects vit backbone and decoder_heads.)
Decoder	Converts features into pixel-level predictions	UNet decoder, SegFormer decoder, UperNet decoder

TaskModel (LightningModule)



Losses, Optimizers, Schedulers

Component	Purpose	Examples
Losses	Guides learning	CrossEntropy, Dice, Focal, hybrid losses
Optimizers	Updates weights	AdamW, SGD
Schedulers	Controls learning rate over time	CosineAnnealing, OneCycle, StepLR

GDL Design Philosophy

Modular: encoder / neck / decoder are swappable

Composable: DOFA, SegFormer, UNet++ share the same interface

Configurable: losses, optimizers, schedulers via YAML

Metadata-aware: temporal, spatial, spectral support for EO

Unified: same interface for training + testing

Reproducible: Lightning + MLflow handle logs & checkpoints

Scalable: multi-GPU, multi-node, multi-sensor ready

Trainer and LightningCLI

The trainer handles the following:

Functions	What it does
fit()	Full training loop
test()	Evaluate best checkpoint after training.
accelerator	GPU/CPU selection
strategy	DDP/Multi-GPU logic
callbacks	Checkpointing, logging, visualization
loggers	MLFlow integration
metrics logging	During train/val/test
gradient accumulation	Efficient multi-GPU
precision control	Mixed precision (fp16/bf16)

Trainer and LightningCLI

LightningCLI sits above the Trainer and orchestrates the workflow by wiring together:

- Dataset
- DataModule
- Model
- Trainer configs
- Logging
- Checkpointing

Using YAML configuration.

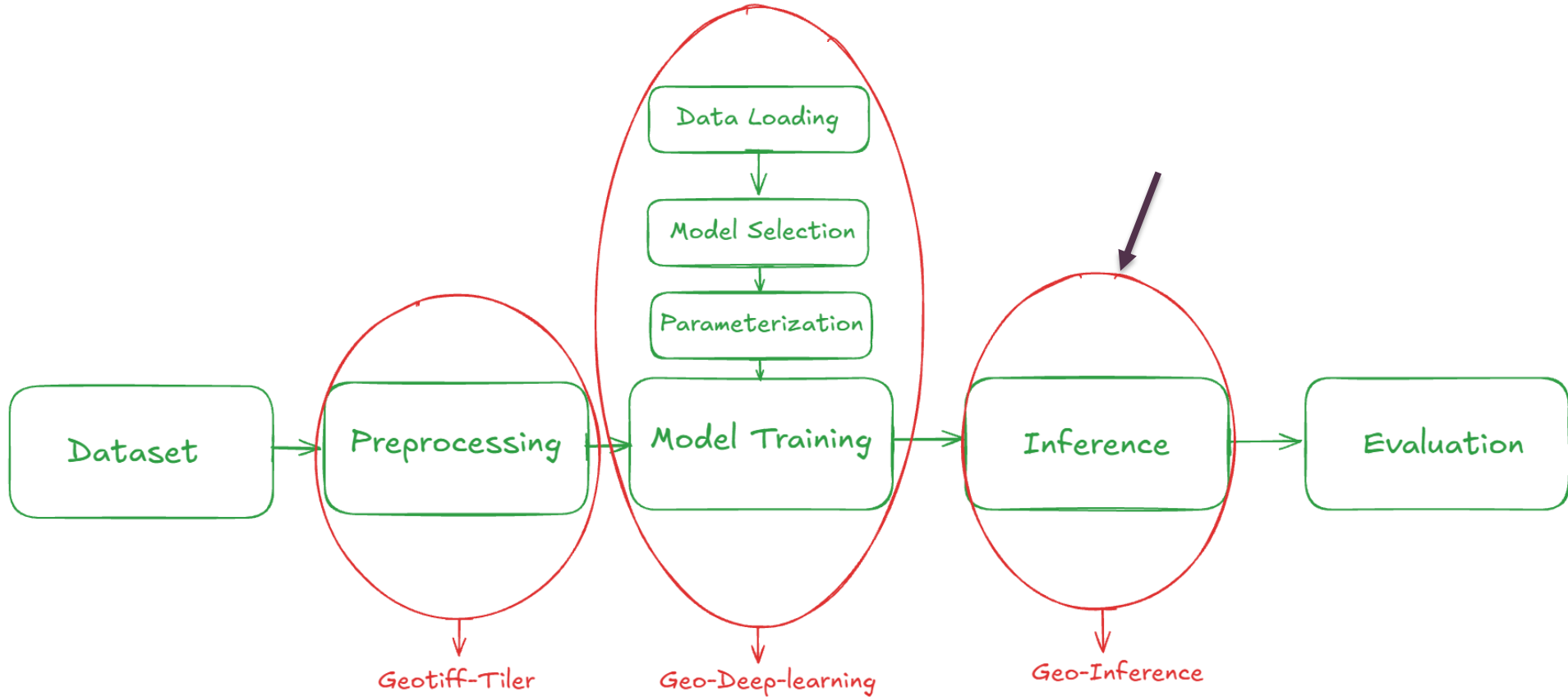
```
trainer:
  max_epochs: 50
  accelerator: gpu
  devices: 4
  precision: bf16
  strategy: ddp

model:
  encoder: mit_b2
  num_classes: 6
  optimizer:
    class_path: torch.optim.AdamW
    init_args:
      lr: 1e-4

data:
  class_path: geo_deep_learning.datamodules.MultiSensorDataModule
  init_args:
    sensor_configs_path: configs/sensors.yaml
    batch_size: 16
```

Inference (geo-inference)

GeoAI Overview



Inference

geo-inference

- Prediction at scale
- Scripted weights (The model ships as a self-contained file)
 - geo-inference never needs to know what architecture
- Parallel Processing
- Reusable across multiple interface (Webapp, QGIS Plugin, e.t.c.)

geo-inference

Setup

- Load TorchScript model
- TTA wrapper (flip/rotate)
- Select device (CPU/GPU)
- Work dir + output paths

Input Loading

- GeoTIFF / Zarr / STAC
- Band reorder (requested)
- Nodata -> valid mask
- BBox windowing (optional)

Chunk & Pad

- Pad to patch multiple
- Dask rechunk (stride)
- map_overlap, depth=stride
- Threaded scheduler

Windowed Inference

- Run model per chunk
- Hann window (edge-aware)
- 9-position window logic
- num_classes+1 channels

Overlap-Add Merge

- Sum overlapping quadrants
- Normalize by weight sum
- Argmax / threshold
- uint8 class mask

Output & Export

- Write raster (nodata=255)
- Mask -> polygons (GeoJSON)
- COCO / YOLO export
- Async GC every 5s



Questions?

Canada